

Part E – relearn on a learned threshold

A deviation that leaves a threshold in LEARNED mode must launch a relearn, on the following flight hours only. The same applies when that deviation is deleted and the threshold returns to LEARNED. Six edits across three files. Every one is given as the exact text to find and the text to put in its place.

What the requirement says

Synthesis document, *Deviation creation* rules: “A deviation set on a learned threshold triggers an automatic relearn request.” Slide 8: “A deviation is not deleted by a relearn request.” Slide 15: the relearn is *sur les heures suivantes* – following hours only, so no past-flight selection and no applicable date.

Slide 17 gives the matrix, and it covers deletion as well as creation:

Deviation	On create	On delete
FIXED → LEARNED	relearn	–
LEARNED → LEARNED, bounds changed	relearn	relearn
LEARNED → FIXED	–	relearn

Every row reduces to one sentence: **launch a relearn whenever the threshold is LEARNED after the change**. On create that is the deviated type; on delete it is the configuration type, because the deviation is gone.

Benoit's note of 30-09 pins down the detail:

He says	Means
if the indicator is LEARNED (type learned <i>or modified from fixed to learned</i>)	Both cases, including a deviation that turns a FIXED threshold into a learned one – see <i>Fixed to learned</i> at the end
Programmatically launch the relearn request without showing the popup	No Threshold Learning dialog; the request is built in code
Description: fill with “New deviation - TE-2026xxxxx”	The TE reference of that deviation goes in the text, so the description differs per deviation
No Flight Hours (to set the learning on future flights)	historyLearningTime = 0

One relearn per deviation, not one per Apply. Because the description carries the TE reference, `createRelearnThresholdRequests` has to be called once for each deviation rather than once for the whole list. That is why the launcher below takes a description and is called inside the loop.

History learning time is zero. `RelearnThresholdRequest.historyLearningTime = 0` makes `LearnedThresholdAnalyzer` skip the *flightsBefore* loop entirely, which is exactly what slide 15 describes.

Where it goes, and what to change

`HealthCommonBusiness` already holds both call sites and already has `createRelearnThresholdRequests` on the same interface, which runs the whole server path including the data-collection record. Reusing it means no new service method and no new DAO code.

Two values have to reach those two methods and currently do not: the aircraft id and the login name. The description is not one of them – it is built from the deviation's own TE reference, so it belongs in the business layer beside the rest of the mapping.

Edit	File	What
1	<code>IHealthCommonBusiness</code>	Two new parameters on two methods
2	<code>HealthCommonBusiness</code>	The helper that finds the learned thresholds, and the launcher
3	<code>HealthCommonBusiness</code>	<code>createDeviations</code> – relearn on create
4	<code>HealthCommonBusiness</code>	<code>acknowledgeDeviations</code> – relearn on delete
5	<code>HealthAlarmsPreferenceStore</code>	Pass aircraft and login through
6	<code>DeviationTableComposite</code>	Give the Delete button the aircraft

Edit 1 — IHealthCommonBusiness

`mgs-business/.../business/health/api/IHealthCommonBusiness.java`

FIND the two declarations (search for `createDeviations`):

```
void createDeviations(List<IndicatorDeviationDto> dtos);

void acknowledgeDeviations(List<String> deviationIds);
```

REPLACE WITH:

```
void createDeviations(String aircraftId, String loginName, List<IndicatorDeviationDto> dtos)
    throws EntityNotFoundException, ConcurrentAccessException;

void acknowledgeDeviations(String aircraftId, String loginName, List<String> deviationIds)
    throws EntityNotFoundException, ConcurrentAccessException;
```

If the current declarations already carry a `throws` clause, keep it and add the two above to it. Both exceptions come from `createRelearnThresholdRequests`, which is declared four methods further up in the same interface.

Edit 2 — HealthCommonBusiness: the helper

FIND the closing brace of acknowledgeDeviations, which is immediately followed by toDtos:

```
private List<IndicatorDeviationDto> toDtos(List<IndicatorDeviation> deviations) {
```

INSERT ABOVE IT, these two methods:

```
/**
 * BCR-6605 the learned thresholds that must be relearned because of this deviation.
 *
 * @param deviatedType
 *      the threshold type after the change: the deviation's type when it set one,
 *      null to mean "whatever the configuration says" - used when the deviation is
 *      deleted, and when it did not change the type
 */
private List<String> getLearnedThresholdIds(IndicatorConfiguration indicatorConfiguration,
String severity, String deviatedType) {
    List<String> ids = new ArrayList<>();
    if (indicatorConfiguration == null) {
        return ids;
    }
    if (deviatedType != null && !ThresholdType.LEARNED.name().equals(deviatedType)) {
        // The deviation turns the threshold into a fixed one: nothing to learn
        return ids;
    }
    for (ThresholdConfiguration threshold : indicatorConfiguration.getThresholdConfigurations()) {
        if (!(threshold instanceof LearnedThresholdConfiguration)) {
            // Only a configured learned threshold can carry a relearn request
            continue;
        }
        if (severity != null
            && !severity.equals(threshold.getThreshold().getSeverity().name())) {
            continue;
        }
        ids.add(threshold.getId());
    }
    return ids;
}

/**
 * BCR-6605 launch the relearn for one deviation, on the following flight hours only.
 * The description carries that deviation's TE reference, so this is called once per
 * deviation rather than once for the whole Apply.
 */
private void launchRelearn(String aircraftId, String loginName, List<String> thresholdIds,
String description) throws ConcurrentAccessException {
    if (thresholdIds.isEmpty()) {
        return;
    }
    // History learning time is 0: the learning uses the following flight hours only
    createRelearnThresholdRequests(aircraftId, thresholdIds, 0L, description, loginName);
}
```

The id is the threshold configuration id. getId() on the LearnedThresholdConfiguration itself, not getThreshold().getId(). This matches what the existing Learn Threshold path passes: HealthTreeViewNodeLearnThresholdAction.learnThreshold() builds its learnedThresholdIds with threshold.getId() over a list of LearnedThresholdConfiguration. The wrong one fails silently – no error, just no relearn row.

No new imports. ArrayList, List, ThresholdType, ThresholdConfiguration, LearnedThresholdConfiguration, IndicatorConfiguration and ConcurrentAccessException are all already imported in this file.

Edit 3 — HealthCommonBusiness: relearn on create

FIND the whole method (the first two lines and the last three are enough to locate it):

```
@Override
public void createDeviations(List<IndicatorDeviationDto> dtos) {
    for (IndicatorDeviationDto dto : dtos) {
        IndicatorDeviation d = new IndicatorDeviation(UUID.randomUUID().toString());
        d.setIndicatorConfiguration(healthCommonService.findIndicatorConfiguration(dto.getIndicatorConfigurationId()));
        ...
        d.setNbSuccessivePoints(dto.getNbSuccessivePoints());
        healthCommonService.createDeviation(d);
    }
}
```

REPLACE WITH:

```
@Override
public void createDeviations(String aircraftId, String loginName, List<IndicatorDeviationDto> dtos)
throws EntityNotFoundException, ConcurrentAccessException {
    for (IndicatorDeviationDto dto : dtos) {
        // BCR-6605 hoisted into a local so the relearn lookup can reuse it
        IndicatorConfiguration indicatorConfiguration =
            healthCommonService.findIndicatorConfiguration(dto.getIndicatorConfigurationId());

        IndicatorDeviation d = new IndicatorDeviation(UUID.randomUUID().toString());
        d.setIndicatorConfiguration(indicatorConfiguration);
        d.setFilteringLaw(healthCommonService.findFilteringLaw(dto.getFilteringLawId()));
        d.setTeReference(dto.getTeReference());
        d.setStartDate(dto.getStartDate());
        d.setEndDate(null);
        d.setSeverity(dto.getSeverity());
        d.setThresholdLevel(dto.getThresholdLevel());
        d.setThresholdType(dto.getThresholdType());
        d.setThresholdValue(dto.getThresholdValue());
```

```

        d.setMinValue(dto.getMinValue());
        d.setMaxValue(dto.getMaxValue());
        d.setLearningTime(dto.getLearningTime());
        d.setMinimumDeviation(dto.getMinimumDeviation());
        d.setMeanFactor(dto.getMeanFactor());
        d.setDeviationCoefficient(dto.getDeviationCoefficient());
        d.setNbSuccessivePoints(dto.getNbSuccessivePoints());
        healthCommonService.createDeviation(d);

        // BCR-6605 the threshold is learned after this deviation, so relearn it.
        // The description carries this deviation's TE, so launch inside the loop.
        launchRelearn(aircraftId, loginName,
            getLearnedThresholdIds(indicatorConfiguration, dto.getSeverity(),
                dto.getThresholdType()),
            RELEARN_DESCRIPTION_CREATE + dto.getTeReference());
    }
}

```

Only two things changed in the body: `findIndicatorConfiguration` moved into a local so it can be used twice, and the `launchRelearn` call at the bottom of the loop. The description comes out as `New deviation - TE-2026-EC155-00010`, which is the format Benoit asked for.

And ADD the two constants at the top of the class, beside the fields:

```

/** BCR-6605 relearn description prefixes. */
private static final String RELEARN_DESCRIPTION_CREATE = "New deviation - ";
private static final String RELEARN_DESCRIPTION_DELETE = "Deviation deleted - ";

```

Benoit specified the create text. He said nothing about the delete side, so `Deviation deleted - TE-...` is a guess that follows the same shape – change the constant if he wants something else. It is one line.

Edit 4 — HealthCommonBusiness: relearn on delete

mgs-business/.../business/health/impl/HealthCommonBusiness.java

FIND:

```

@Override
public void acknowledgeDeviations(List<String> deviationIds) {
    Date now = new Date();
    for (String id : deviationIds) {
        healthCommonService.acknowledgeDeviation(id, now);
    }
}

```

REPLACE WITH:

```

@Override
public void acknowledgeDeviations(String aircraftId, String loginName, List<String> deviationIds)
    throws EntityNotFoundException, ConcurrentAccessException {
    // BCR-6605 read them while they are still active: once closed they leave this list
    List<IndicatorDeviation> closing = new ArrayList<>();
    for (IndicatorDeviation deviation
        : healthCommonService.findActiveDeviations(getIndicatorIds(aircraftId))) {
        if (deviationIds.contains(deviation.getId())) {
            closing.add(deviation);
        }
    }

    Date now = new Date();
    for (String id : deviationIds) {
        healthCommonService.acknowledgeDeviation(id, now);
    }

    // BCR-6605 the threshold reverts to the configuration, so relearn if that one is learned.
    // deviatedType is null on purpose: the deviation is going away.
    for (IndicatorDeviation deviation : closing) {
        launchRelearn(aircraftId, loginName,
            getLearnedThresholdIds(deviation.getIndicatorConfiguration(),
                deviation.getSeverity(), null),
            RELEARN_DESCRIPTION_DELETE + deviation.getTeReference());
    }
}

```

There is no single-deviation lookup on the service, so this reuses `findActiveDeviations` – the same call the Deviation List tab already makes. One extra query per Delete over a handful of rows; the synthesis document puts the total at three or four per aircraft.

Edit 5 — HealthAlarmsPreferenceStore

mgs-client/.../preferences/healthalarms/store/HealthAlarmsPreferenceStore.java

Two places. First, `saveDeviations` needs to remember which aircraft it is working on.

FIND:

```

public void saveDeviations() {
    List<IndicatorDeviationDto> newDeviations = new ArrayList<>();
    Date now = new Date();
    for (Entry<Aircraft, WCMechanicalComponentModel> entry : data.entrySet()) {
        Multimap<Indicator, WCThresholdModel> thresholds = getComponentThresholds(entry.getValue());
    }
}

```

REPLACE WITH:

```

public void saveDeviations() {
    List<IndicatorDeviationDto> newDeviations = new ArrayList<>();
    Aircraft deviationAircraft = null;    // BCR-6605
    Date now = new Date();
    for (Entry<Aircraft, WCMechanicalComponentModel> entry : data.entrySet()) {
    }
}

```

```
deviationAircraft = entry.getKey(); // BCR-6605
Multimap<Indicator, WCThresholdModel> thresholds = getComponentThresholds(entry.getValue());
```

Then **FIND**, at the end of the same method:

```
if (!newDeviations.isEmpty()) {
    try {
        healthBusiness.createDeviations(newDeviations);
    } catch (Exception e) {
        LOGGER.error("Could not save deviations", e);
    }
}
```

REPLACE WITH:

```
if (!newDeviations.isEmpty() && deviationAircraft != null) {
    try {
        healthBusiness.createDeviations(deviationAircraft.getId(),
            MgsSession.getInstance().getUser().getLoginName(), newDeviations);
    } catch (Exception e) {
        LOGGER.error("Could not save deviations", e);
    }
}
```

Second, the acknowledge wrapper, further down the same file.

FIND:

```
public void acknowledgeDeviations(List<String> deviationIds) {
    try {
        healthBusiness.acknowledgeDeviations(deviationIds);
    } catch (Exception e) {
        LOGGER.error("Could not acknowledge deviations", e);
    }
}
```

REPLACE WITH:

```
public void acknowledgeDeviations(Aircraft aircraft, List<String> deviationIds) {
    if (aircraft == null) {
        return;
    }
    try {
        healthBusiness.acknowledgeDeviations(aircraft.getId(),
            MgsSession.getInstance().getUser().getLoginName(), deviationIds);
    } catch (Exception e) {
        LOGGER.error("Could not acknowledge deviations", e);
    }
}
```

No new imports: `Aircraft` and `MgsSession` are already imported in this file.

`getLoginName()` is the right name – it is what `HealthTreeNodeLearnThresholdAction` passes for the same parameter.

Edit 6 — DeviationTableComposite

mgs-client/.../preferences/healthalarms/DeviationTableComposite.java

The Delete Deviation button calls the wrapper that just gained a parameter.

FIND the call inside the Delete button's listener (search for `acknowledgeDeviations`):

```
store.acknowledgeDeviations(ids);
```

REPLACE WITH:

```
store.acknowledgeDeviations(MgsSession.getInstance().getAircraft(), ids);
```

If the composite already holds the aircraft as a field, use that instead of the session – it is the same aircraft either way. Add the `MgsSession` import if it is not there.

While you are in HealthCommonBusiness

`toDtos` has an unrelated NPE. The filtering-law parameters loop sits outside the null check immediately above it.

FIND:

```
if (d.getFilteringLaw() != null) {
    dto.setFilteringLawId(d.getFilteringLaw().getId());
    dto.setFilteringLawName(d.getFilteringLaw().getName());
}
StringBuilder params = new StringBuilder();
for (ContextualParameter p : d.getFilteringLaw().getContextualParameters()) {
```

REPLACE WITH:

```
StringBuilder params = new StringBuilder();
if (d.getFilteringLaw() != null) {
    dto.setFilteringLawId(d.getFilteringLaw().getId());
    dto.setFilteringLawName(d.getFilteringLaw().getName());
    for (ContextualParameter p : d.getFilteringLaw().getContextualParameters()) {
```

Then close the extra brace at the end of that loop. A deviation created without a filtering law throws today when the Deviation List tab loads.

Build and test

#	Action	Expect
1	Rebuild mgs-business, mgs-client and the server, then redeploy	–
2	Find an indicator whose threshold is LEARNED. Create a deviation on it changing the min or max, add a TE, Apply and OK	Row turns green
3	SELECT * FROM public.relearn_threshold_request ORDER BY startdate DESC LIMIT 5;	A new row, handled = false, history_learning_time = 0
4	Check the deviation is still listed in the Deviation List tab	Still there – the relearn must not remove it (slide 8)
5	Download flights until the learning time is reached	handled = true and learnedvalue filled on that row
6	Delete the deviation from the Deviation List tab	Moves to history, and a second relearn row appears
7	Create a deviation on a FIXED threshold, leaving the type alone	No new relearn row

Step 3 proves the feature. Step 7 is what separates a correct implementation from one that launches a relearn on every deviation regardless of threshold type.

Fixed to learned – the one part that is not covered above

Benoit's note includes "*or modified from fixed to learned*", and the six edits above do not handle it. This is not an oversight in the edits: the data model has nowhere to put the request.

relearnRequests is declared on LearnedThresholdConfiguration, not on the ThresholdConfiguration base class:

```
@OneToMany(cascade = CascadeType.ALL, fetch = FetchType.EAGER, orphanRemoval = true)
@JoinColumn(name = "THRESHOLD_CONFIGURATION_ID")
private Set<RelearnThresholdRequest> relearnRequests = new HashSet<>();
```

When a deviation turns a FIXED threshold into a learned one, the configuration row is still a FixedThresholdConfiguration. There is no collection to add the request to, and HealthDao.createRelearnThresholdRequest takes a LearnedThresholdConfiguration parameter, so it cannot be called either. The helper in edit 2 skips the case deliberately rather than failing at runtime.

Two ways to close it:

Option	What it takes
A. Move relearnRequests up to ThresholdConfiguration	The join column is already THRESHOLD_CONFIGURATION_ID and the hierarchy shares one table, so the JPA move itself is small. But LearnedThresholdAnalyzer.updateLearnedThresholds only walks instanceof LearnedThresholdConfiguration, so it would never process the request – that loop has to read the deviation too, which means the deviation map from Part D reaching LearnedThresholdAnalyzer as well.
B. Compute it at analysis time from the deviation's own parameters	No relearn row at all for this case. RedAmberAlarmAnalyzer already has the deviation and its learningTime, meanFactor, deviationCoefficient and minimumDeviation. Larger change, but it keeps the deviation as the single source of truth and avoids a request that points at a fixed configuration.

Either is a day or two, not an afternoon, and both touch mgs-download. Worth settling the scope before starting: the five other cells of the slide 17 matrix work with the six edits above, so this can ship separately.

On calling computeThresholdLearning directly

Benoit's follow-up points at LearnedThresholdAnalyzer.computeThresholdLearning. That method is private, lives in mgs-download, and takes a Flight, a HealthMonitoringParameters and an IHealthHistoryProvider – none of which exist in the business layer at the moment a deviation is created. It also could not produce a value there: with historyLearningTime = 0 and no future flights yet, there is nothing to learn from.

The reading that fits both of his messages is that the request must reach that method with the right parameters, which is what the six edits do – updateLearnedThresholds calls computeThresholdLearning for the latest unhandled request on every download. If he means something else, it is worth one line back to him before building anything.